PostgreSQL — Complete Notes

From Beginner to Advanced

What is PostgreSQL?

PostgreSQL (also called **Postgres**) is a powerful, open-source **Relational Database Management System (RDBMS)**.

- It is based on SQL (Structured Query Language).
- It was earlier known as **POSTGRES**, developed at UC Berkeley.
- PostgreSQL supports **ACID** (Atomicity, Consistency, Isolation, Durability) transactions.
- It is used to store, retrieve, and manage data reliably.

Ex:- Used in web apps, financial systems, data warehousing, etc.

Installation of PostgreSQL on Linux (Ubuntu/Debian)

Step 1 — Update the system

```bash
sudo apt update
sudo apt upgrade
```

Step 2 — Install PostgreSQL

```bash
sudo apt install postgresql postgresql-contrib
```

Step 3 — Check status

```bash
sudo systemctl status postgresql
```

Step 4 — Start / Stop / Restart

```bash
sudo systemctl start postgresql
sudo systemctl stop postgresql
sudo systemctl restart postgresql
```

Step 5 — Access PostgreSQL shell

```bash
sudo -i -u postgres
psql
```

Or in one command:

```bash
sudo -u postgres psql
```

Step 6 — Exit the shell

```sql
\q
```

Step 7 — Set password for postgres user

```sql
ALTER USER postgres PASSWORD 'yourpassword';
```

PostgreSQL vs MySQL — Key Differences

Feature	MySQL	PostgreSQL
Type	Relational	Object-Relational
Compliance	Partial SQL	Full SQL Standard
JSON Support	Basic	Advanced (JSONB)

Feature	MySQL	PostgreSQL
Inheritance	No	Yes (Table Inheritance)
Arrays	No	Yes
Full-text Search	Basic	Advanced
Concurrency	MVCC (limited)	Full MVCC

psql — Command Line Tool

`psql` is the interactive terminal for PostgreSQL.

Useful psql Commands :-

Command	Purpose
`\l`	List all databases
`\c dbname`	Connect to a database
`\dt`	List all tables
`\d tablename`	Describe a table
`\du`	List all users/roles
`\q`	Quit psql
`\i file.sql`	Run a SQL file
`\timing`	Toggle query timing
`\e`	Open editor for query

Data :-

Data is defined as facts or figures, or information that is stored in or used by a computer.

Ex:- Information collected for a research paper, an email etc.

Database :-

Database is nothing but an organized form of data for easy access, storing, retrieval and managing of data. This is also known as structured form of data which can be accessed in many ways.

Ex:- School Management Database, Bank Management Database.

DBMS :- (Data Base Management System)

It is a program that controls creation, maintenance and use of a database. DBMS can be termed as a file manager that manages data in a database rather than saving it to file systems.

RDBMS :- (Relational Database Management System)

RDBMS stores the data into the collection of tables, which is related by common fields between the columns of the table. It also provides relational operators to manipulate the data stored into the tables.

Ex:- PostgreSQL, SQL Server.

Tables & Fields :-

A table is a set of data that are organized in columns and rows. Columns can be categorized as Vertical, and Rows are horizontal. A table has specified number of columns called fields but can have any number of rows which is called records.

Ex:- Table Name: Employee.

- Fields:- EmpID, Emp Name, Date of Birth.
- Data:- 201456, David, 11/15/2000

EMP ID	ADDRESS	SALARY
Ankit	Lucknow	15000
Raman	Allahabad	18000
Mike	New York	20000

SQL :-

- SQL stands for Structured Query Language.
- SQL was earlier known as SEQUEL.
- It is used to communicate with the database.
- This is a standard language used to perform tasks such as retrieval, updation, insertion and deletion of data from a database.

Types of SQL Commands :-

SQL categorizes its commands on the basis of functionalities performed by them. There are five types of SQL commands which can be classified as :-

1. Data Definition Language (DDL) :- `[ CREATE, ALTER, DROP, TRUNCATE ]` → Allows to create data structures i.e tables, delete, alter etc.

2. Data Manipulation Language (DML) :- `[ INSERT, UPDATE, DELETE ]` → Allows modification of data in the database.

3. Data Query Language (DQL) :- `[ SELECT, SHOW, HELP ]` → Helps in retrieving data and information from the database.

4. Data Control Language (DCL) :- `[ GRANT, REVOKE ]` → Allows or denies permissions to access the tables.

5. Transaction Control Language (TCL) :- `[ COMMIT, ROLLBACK, SAVEPOINT ]` → Manages the changes made by the DML statements.

Operators :-

An operator is a reserved character or a word which is used in SQL statement to query our database. We use a WHERE clause to query a database using operators. Operators are needed to specify conditions in a SQL statement.

Types of Operators —

1. **Arithmetic Operators** → `+, -, *, /, %`
2. **Comparison Operators** → `<, >, =, !=, <=, >=`
3. **Logical Operators** → `AND, OR, NOT, BETWEEN, IN, ANY, ALL, LIKE`

Data Types in PostgreSQL :-

→ Data types are used to represent the nature of the data that can be stored in the database table.
→ Each column in a database table is required to have a name and a data type.

1. Numeric :-

Type	Size	Description
`SMALLINT`	2 bytes	Small integers (-32768 to 32767)
`INTEGER` or `INT`	4 bytes	Standard integers
`BIGINT`	8 bytes	Large integers
`DECIMAL(m,d)`	Variable	Exact decimal
`NUMERIC(m,d)`	Variable	Exact numeric
`REAL`	4 bytes	Float (6 decimal digits)
`DOUBLE PRECISION`	8 bytes	Float (15 decimal digits)
`SERIAL`	4 bytes	Auto-increment integer
`BIGSERIAL`	8 bytes	Auto-increment big integer

2. Date & Time :-

Type	Size	Format
`DATE`	4 bytes	YYYY-MM-DD
`TIME`	8 bytes	HH:MM:SS
`TIMESTAMP`	8 bytes	YYYY-MM-DD HH:MM:SS
`TIMESTAMPTZ`	8 bytes	Timestamp with timezone
`INTERVAL`	16 bytes	Time span

3. String :-

Type	Description
`CHAR(n)`	Fixed length (n characters)
`VARCHAR(n)`	Variable length (up to n characters)
`TEXT`	Unlimited length string

4. Boolean :-

`BOOLEAN` — stores `TRUE`, `FALSE`, or `NULL`.

5. PostgreSQL Special Types :-

> These are NOT in MySQL — PostgreSQL superpower!

Type	Description
`JSON`	Stores JSON data
`JSONB`	Binary JSON (faster queries)
`ARRAY`	Column can store arrays
`UUID`	Universally Unique Identifier
`INET`	IP address
`CIDR`	Network address
`HSTORE`	Key-value pairs
`TSVECTOR`	Full-text search
`POINT, LINE, CIRCLE`	Geometric types
`BYTEA`	Binary data

Creating a Database :-

```sql
```

```sql
CREATE DATABASE database_name;
```

Ex:-

```sql
CREATE DATABASE school;
```

Connect to a database :-

```sql
\c school
```

Drop a database :-

```sql
DROP DATABASE database_name;
```

Data Definition Language (DDL) :-

→ DDL consists of the SQL commands that can be used to define the database schema.
→ It is a set of SQL commands used to create, modify and delete database structures but not data.

CREATE :-

→ Used to create tables or databases.

Syntax:-

```sql
CREATE TABLE table_name (
    col1 datatype constraint,
    col2 datatype,
    ...
);
```

Ex:-

```sql
CREATE TABLE students (
    ID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT NOT NULL
);
```

ALTER :-

→ Used to modify the values in the tables.

1. ALTER TABLE ADD column :- Used to add a column.

Syntax:-

```sql
ALTER TABLE table_name ADD col_name column_definition;
```

Ex:-

```sql
ALTER TABLE student ADD marks INT;
```

To add multiple columns:-

```sql
ALTER TABLE table_name ADD col1, ADD col2;
```

2. ALTER TABLE MODIFY column :- To modify the column in a table.

Syntax:-

```sql
ALTER TABLE table_name ALTER COLUMN col_name TYPE new_datatype;
```

Ex:-

```sql
```

```sql
ALTER TABLE student ALTER COLUMN Name TYPE VARCHAR(40);
```

3. ALTER TABLE RENAME column :- To rename a column name.

Syntax:-

```sql
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
```

Ex:-

```sql
ALTER TABLE students RENAME COLUMN first_Name TO student_name;
```

4. ALTER TABLE DROP column :- (To delete column)

Syntax:-

```sql
ALTER TABLE table_name DROP COLUMN col_name;
```

Ex:-

```sql
ALTER TABLE students DROP COLUMN address;
```

RENAME Table Command :-

Syntax:-

```sql
ALTER TABLE old_table_name RENAME TO new_name;
```

Ex:-

```sql
ALTER TABLE students RENAME TO student_details;
```

TRUNCATE TABLE Command :-

→ A Truncate SQL command is used to remove all rows (complete data) from the table.
→ It doesn't delete the table structure, just empties the table.

Syntax:-

```sql
TRUNCATE TABLE table_name;
```

DROP TABLE Command :-

→ DROP TABLE statement is used to delete a table definition and all the data from the table.

Syntax:-

```sql
DROP TABLE table_name;
```

Constraints in PostgreSQL :-

→ Constraints are certain conditions, rules or restrictions we apply on the database.
→ Constraints could be either on a column level or table level.

1. NOT NULL :-

NULL → Empty (no value is not available). Whenever the table's column is declared as NOT NULL, then the value for that column cannot be empty for any of the table's records.

Syntax:-

```sql
CREATE TABLE TableName (
    Column1 datatype NOT NULL,
    Column2 datatype NOT NULL, ...
);
```

Ex:-

```sql
CREATE TABLE student (
    StudentID INT NOT NULL,
    Student_fName VARCHAR(20),
    LName VARCHAR(20)
);
```

2. UNIQUE :-

→ Duplicate values are not allowed in the columns to which the UNIQUE constraint is applied.
→ This constraint can be applied to one or more than one column in a table.

Syntax:-

```sql
CREATE TABLE students (StudentID INT UNIQUE, ... )
CREATE TABLE TableName (col1 dtype UNIQUE, col2, ...)
```

3. PRIMARY KEY :-

→ Primary key constraint is a combination of NOT NULL and UNIQUE constraints.
→ There should be only one primary key column in a table.
→ Primary key becomes foreign key in other tables, when creating relations among tables.

Syntax:-

```sql
CREATE TABLE TableName (col1 dtype PRIMARY KEY, ...)
```

Ex:-

```sql
CREATE TABLE Students (student_id INT PRIMARY KEY, ...)
```

4. FOREIGN KEY :-

→ A foreign key in one table points to a primary key in another table. It is a referential constraint between two tables.

Syntax:-

```sql
```

```sql
CREATE TABLE tableName (
    col1 dtype PRIMARY KEY,
    col2 dtype,
    FOREIGN KEY (colName) REFERENCES Parent_table_name (primary_key_col_name)
);
```

Ex:-

```sql
CREATE TABLE parent_table (id INT PRIMARY KEY, name VARCHAR(255));

CREATE TABLE child_table (
    id INT PRIMARY KEY,
    parent_id INT,
    FOREIGN KEY (parent_id) REFERENCES parent_table (id)
);
```

5. CHECK :-

Whenever a check constraint is applied to the table's column, and the user wants to insert the value in it, then the value will first be checked for certain conditions, before inserting the value into that column.

Syntax:-

```sql
CREATE TABLE TableName (col1 dtype CHECK (col1 condition), col2 ...);
```

Ex:-

```sql
CREATE TABLE Students (Age INT CHECK (Age <= 15));
```

6. DEFAULT :-

Whenever default constraint is applied to a column, then if the user doesn't give any value to it then default value is inserted to that record.

Syntax:-

```sql
```

```
CREATE TABLE table_name (col1 dtype DEFAULT value, ...)
```

Ex:-

```sql
CREATE TABLE students (Branch varchar(20) DEFAULT 'ECE');
```

Data Modification Language (DML) :-

Once the tables are created and database is generated using DDL commands, manipulation inside those tables and databases is done using DML commands.

INSERT Command :-

→ It is used to insert a single or multiple records in a table.
→ We can use Insert command in two syntaxes.

Syntax-1:-

```sql
INSERT INTO Table_name (col1, col2, col3 ...)
VALUES (val1, val2, val3, ...);
```

Ex:-

```sql
INSERT INTO students (ID, name, age)
VALUES (101, 'Venty', 23);
```

Syntax-2:-

```sql
INSERT INTO Table_name VALUES (val1, val2, ...);
```

Ex:-

```sql
```

```sql
INSERT INTO students VALUES (102, 'Rocky', 25);
```

To insert multiple rows:-

```sql
INSERT INTO table_name (col1, col2)
VALUES (1, 2), (3, 4), (5, 6);
```

UPDATE Command :-

→ Update statement is used to update the existing data in a table.
→ We can use the Update statement to change column values of a single row, a group of rows, or all rows in a table.

Syntax:- To update all rows in a table:-

```sql
UPDATE table_name SET col1 = expr1, col2 = expr2;
```

Ex:-

```sql
UPDATE students SET city = 'Hyderabad';
```

Syntax:- To update a particular record/row:-

```sql
UPDATE table_name SET col_1 = expr1, col_2 = expr2
WHERE condition;
```

Ex:-

```sql
UPDATE students SET marks = 50 WHERE ID = 102;
```

DELETE Command :-

→ The Delete statement is used to delete rows from a table.
→ Generally Delete statement removes one or more records from a table.

Syntax:-

```sql
DELETE FROM table_name WHERE condition;
```

Ex:-

```sql
DELETE FROM students WHERE ID = 101;
```

To delete all the records from the table:-

```sql
DELETE FROM table_name;
```

Difference b/w DELETE and TRUNCATE statements:

→ DELETE statement only deletes the rows from the table based on the condition defined by WHERE clause or delete all the rows from table when condition is not specified. But it doesn't free the space containing by the table.
→ TRUNCATE is used to delete all the rows from the table and free the containing space.

Data Query Language (DQL) :-

→ Data query language consists of command over which data selection in SQL relies.
→ SELECT command in combination with other SQL clauses is used to retrieve and fetch data from database/tables on the basis of certain conditions applied by user.
→ Resultant table is called as result-set table.

Syntax :-

To select all attributes (columns) and tuples (rows) from a table:

```sql
SELECT * FROM table_name;
```

To select few attributes and all tuples from a table:

```sql
SELECT col_1, col_2, col_3 FROM table_name;
```

Select DISTINCT:

The SQL DISTINCT command is used with SELECT keyword to retrieve only distinct or unique data.

Syntax:-

```sql
SELECT DISTINCT col_1, col_2 FROM table_name;
```

Ex:-

```sql
SELECT DISTINCT city FROM students;
```

WHERE Clause :-

→ The WHERE clause is used to filter records based on a condition.
→ It is used in SELECT, UPDATE, DELETE statements.

Syntax:-

```sql
SELECT col_1, col_2, ... FROM table_name WHERE condition;
```

Ex:-

```sql
SELECT ID, NAME, salary FROM customers WHERE salary > 2000;
```

Logical Operators in WHERE clause :-

i. **LIKE** → used for matching or checking. ii. **AND** → used for combining two conditions together. iii. **IN** → used to compare values. iv. **BETWEEN** → used to compare values within a range. v. **OR** →

used to check either of the two comparisons. vi. **NOT** → used to fetch value other than values which are not required.

AND Operator :-

Logical AND operator compares two Boolean expressions and returns TRUE when both of the conditions are TRUE and returns FALSE when either is FALSE.

Syntax:-

```sql
SELECT col_1, col2 FROM table_name WHERE condition1 AND condition2;
```

Ex:-

```sql
SELECT first_name, second_name FROM students WHERE age >= 10 AND age <= 15;
```

OR Operator :-

Logical OR compares two Boolean expressions and returns True when either of the conditions is TRUE and returns False when both are false.

Syntax:-

```sql
SELECT col1, col2, ... FROM table_name WHERE condition1 OR condition2 OR condition3
```

Ex:-

```sql
SELECT Name, roll_no FROM student_details WHERE subject = 'Maths' OR subject = 'Scie
```

NOT Operator :-

→ If we want to find rows that do not satisfy a condition, that is, if a condition is satisfied, then the

row is not returned.

Syntax:-

```sql
SELECT col1, col2... FROM table_name WHERE NOT condition;
```

Ex:-

```sql
SELECT first_name, last_name FROM students_details WHERE NOT games = 'Football';
```

BETWEEN Operator :-

→ This operator displays the records which fall between the given ranges.

Syntax:-

```sql
SELECT column_name(s) FROM table_name WHERE column_name BETWEEN Value1 AND Value2;
```

Ex:-

```sql
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 70000;
```

NOT BETWEEN:-

To display the products outside a specific range.

Ex:-

```sql
SELECT * FROM employees WHERE Salary NOT BETWEEN 50000 AND 70000;
```

IN Operator :-

→ When we want to check for one or more than one value in a single SQL query, for that we use

IN operator.
→ IN operator allows you to determine if a value matches any value in a list of values.

Syntax:-

```sql
SELECT column_name(s) FROM table_name WHERE col_name IN (val1, val2, ... valn);
```

Ex:-

```sql
SELECT * FROM employees WHERE age IN (25, 27, 24);
```

NOT IN operator:-

```sql
SELECT * FROM employees WHERE age NOT IN (25, 27);
```

LIKE Operator :-

→ LIKE operator in SQL displays only those data from the table which matches the pattern specified in the query.
→ There are two wildcards used in conjunction with the LIKE operator.

- `%` → Represents zero, one or multiple characters.
- `_` → Represents a single number or character.

Syntax:-

```sql
SELECT col1, col2, col3, .. FROM table_name WHERE col_name LIKE pattern;
```

Different patterns :-

Pattern	Meaning
`'a%'`	Starts with a
`'%a'`	Ends with a
`'%or%'`	Having 'or' in any position
`'_r%'`	Having 'r' in the second position
`'a_%'`	Starts with 'a' and at least 2 characters in length
`'a__%'`	Starts with 'a' and at least 3 characters in length
`'a%o'`	Starts with 'a' and ends with 'o'

NOT LIKE operator:-

→ It works exactly opposite to the LIKE operator.
→ It is used to access the values that doesn't follow the specific patterns.

Syntax:-

```sql
SELECT * FROM Customers WHERE customer_name NOT LIKE 'a%';
```

NULL Values :-

→ A field with a null value is a field with no value.
→ It is not possible to test for NULL values with comparison operators like =, <, > etc.
→ For that we use 'IS NULL' or 'IS NOT NULL' commands.

IS NULL Syntax:-

```sql
SELECT column_name(s) FROM table_name WHERE column_name IS NULL;
```

Ex:-

```sql
```

```sql
SELECT * FROM students WHERE age IS NULL;
```

IS NOT NULL Syntax:-

```sql
SELECT col_name(s) FROM table_name WHERE col_name IS NOT NULL;
```

Ex:-

```sql
SELECT * FROM students WHERE age IS NOT NULL;
```

LIMIT Command :-

→ It is used to specify the number of records to return.
→ It is useful on large tables with thousands of records, returning a large number of records can impact performance.

Syntax:-

```sql
SELECT col_name(s) FROM table_name LIMIT offset, count;
```

> **PostgreSQL Note:-** In PostgreSQL, use `LIMIT` and `OFFSET` separately.

```sql
SELECT * FROM emp_info LIMIT 7 OFFSET 3;
```

ORDER BY Clause :-

→ It is used to sort the result-set in ascending or descending order.
→ By default it sorts in ascending order.
→ Use DESC keyword to sort in descending order.

Syntax:-

```sql
```

```sql
SELECT col1, col2... FROM table_name ORDER BY col1, col2, ... ASC | DESC;
```

Ex:-

```sql
SELECT * FROM CUSTOMERS ORDER BY NAME;
SELECT * FROM CUSTOMERS ORDER BY NAME DESC;
```

Renaming Attributes or SQL Aliases :-

→ SQL aliases are used to give a table, or a column in a table, a temporary name.
→ Aliases are often used to make column names more readable.
→ An alias only exists for the duration of that query.
→ An alias is created with the AS keyword.

Syntax:-

```sql
SELECT col_name AS alias_name FROM table_name;
```

Ex:-

```sql
SELECT ID as Emp_ID, Name AS emp_name FROM table_name;
```

BUILT-IN SQL Functions :-

→ A function is a set of SQL statements that perform a specific task.
→ Functions provide code reusability.
→ If you have to write large SQL scripts to perform the same task, you can create a function that performs the task.
→ Next time we can call the function instead of writing it.

There are four types of functions in SQL :-

1. String()
2. Math()

3. Date()
4. Aggregate()

String() Functions :-

String methods in SQL are useful for processing the string data type or manipulation of string values.

i. CONCAT :- → Used to add two or more strings.

Syntax:-

```sql
SELECT CONCAT('str1', 'str2', ...);
```

Ex:-

```sql
SELECT CONCAT('xyz', 'abc');
```

ii. LOWER :- This function is used to convert the upper case character into lower case.

Syntax:-

```sql
SELECT LOWER(text);
```

Ex:-

```sql
SELECT LOWER(Name) AS LowercaseName FROM emp_info;
```

iii. UPPER :- It converts the lower case characters into uppercase.

Syntax:-

```sql
SELECT UPPER(text);
```

Ex:-

```sql
SELECT UPPER(Name) AS UpName FROM emp_info;
```

iv. REPLACE :- Used to replace all occurrences of the substring in a specified string with another string value.

Syntax:-

```sql
REPLACE(old_string, new_string);
```

Ex:-

```sql
SELECT REPLACE('good Morning', 'G');
```

v. REVERSE :- Displays characters in a string in reverse order.

Syntax:-

```sql
REVERSE(string);
```

Ex:-

```sql
SELECT REVERSE(Name) AS rev_name FROM emp_info;
```

vi. LENGTH :- It gives number of characters in a string, including trailing spaces.

Syntax:-

```sql
LENGTH(str);
```

Ex:-

```sql
```

```sql
SELECT LENGTH(name) FROM emp_info;
```

vii. SUBSTRING :- This function extracts a substring from a string that begins at a specific position and ends at a specific length.

Syntax:-

```sql
SUBSTRING(str, start, length);
```

Ex:-

```sql
SELECT SUBSTRING(Name, 1, 4) AS extractString FROM emp_info;
```

viii. LTRIM :- It returns a string from a given string after removing leading spaces.

Syntax:-

```sql
LTRIM(string);
```

Ex:-

```sql
SELECT LTRIM('   Hello world') AS LefttrimmedStr;
```

ix. RTRIM :- It returns a string from a given string after removing all trailing spaces.

Syntax:-

```sql
RTRIM('good   ') AS righttrimmedStr;
```

PostgreSQL Extra String Functions :-

> These go beyond what MySQL has!

POSITION :- Returns the position of a substring in a string.

```sql
SELECT POSITION('world' IN 'Hello world');
```

TRIM :- Removes both leading and trailing spaces.

```sql
SELECT TRIM('   Hello   ');
```

INITCAP :- Converts first letter of each word to uppercase. (PostgreSQL only!)

```sql
SELECT INITCAP('hello world');  -- Returns 'Hello World'
```

SPLIT_PART :- Splits a string by a delimiter and returns a part. (PostgreSQL only!)

```sql
SELECT SPLIT_PART('a,b,c', ',', 2);  -- Returns 'b'
```

REPEAT :- Repeats a string N times.

```sql
SELECT REPEAT('abc', 3);  -- Returns 'abcabcabc'
```

LPAD / RPAD :- Pads a string to a given length from left or right.

```sql
SELECT LPAD('5', 3, '0');   -- Returns '005'
SELECT RPAD('Hi', 5, '.');  -- Returns 'Hi...'
```

MATH() Functions :-

→ Math functions are used to perform mathematical calculations.

1. **ABS(x) :-** It gives absolute value of a number.

```sql
```

```sql
SELECT ABS(-6);
```

2. **MOD(x, y) :-** The variable x is divided by y and their remainder is returned.

```sql
SELECT MOD(9, 5);
```

3. **FLOOR(x) :-** Returns the largest integer value that is either less than x or equal to it.

```sql
SELECT FLOOR(25.75);
```

4. **CEILING(x) :-** Returns the smallest integer value that is either more than x or equal to it.

```sql
SELECT CEILING(25.75);
```

5. **TRUNCATE(x, D) :-** Returns the number x, truncated to D decimal places.

```sql
SELECT TRUNC(123.321, 2);
```

6. **EXP(x) :-** Returns e raised to the power of x → e^x.

```sql
SELECT EXP(2);
```

7. **POWER(x, y) :-** Returns x^y.

```sql
SELECT POWER(4, 2);  -- Returns 16
```

8. **SQRT(x) :-** Returns square root of x → √x.

```sql
SELECT SQRT(3);
```

> **PostgreSQL Note:-** Use `TRUNC` instead of `TRUNCATE` for numbers.

Date Functions :-

1. **CURRENT_DATE() :-** Returns the current date.

```sql
SELECT CURRENT_DATE;
```

2. **NOW() :-** Returns the current date and time.

```sql
SELECT NOW();
```

3. **CURRENT_TIMESTAMP :-** Returns the system's current date & time.

```sql
SELECT CURRENT_TIMESTAMP;
```

4. **AGE(date) :-** (PostgreSQL only!) Returns the age interval between two timestamps.

```sql
SELECT AGE('2000-01-01');
SELECT AGE(NOW(), '2000-01-01');
```

5. **DATE_PART :-** (PostgreSQL only!) Extract part of a date.

```sql
SELECT DATE_PART('year', NOW());
SELECT DATE_PART('month', NOW());
SELECT DATE_PART('day', NOW());
```

6. **EXTRACT :-** Same as DATE_PART.

```sql
SELECT EXTRACT(YEAR FROM NOW());
SELECT EXTRACT(MONTH FROM CURRENT_DATE);
```

7. **TO_DATE :-** Converts string to date.

```sql
SELECT TO_DATE('2024-11-30', 'YYYY-MM-DD');
```

8. **TO_CHAR :-** Converts date to formatted string. (PostgreSQL version of DATE_FORMAT)

```sql
SELECT TO_CHAR(NOW(), 'DD-MM-YYYY');
SELECT TO_CHAR(NOW(), 'Month DD, YYYY');
```

9. **DATE_TRUNC :-** Truncates a timestamp to a specified precision. (PostgreSQL only!)

```sql
SELECT DATE_TRUNC('month', NOW());  -- Gives first day of current month
```

Aggregate Functions :-

→ It returns one value after calculating multiple values of a column.
→ Aggregate functions are also called as grouping functions.

1. AVG() :- Calculates average of a set of values.

Syntax:-

```sql
SELECT AVG(col_name) AS alias_name FROM table_name;
```

Ex:-

```sql
SELECT AVG(salary) AS avg_salary FROM emp_info;
```

2. COUNT() :- Returns the number of rows in a database table.

Syntax:-

```sql
SELECT COUNT(salary) FROM emp_info;
SELECT COUNT(col_name) FROM table_name;
```

3. MAX() :- Returns maximum value of the selected column.

Syntax:-

```sql
SELECT MAX(col_name) FROM table_name;
```

Ex:-

```sql
SELECT MAX(salary) FROM emp_info;
```

4. MIN() :- Returns minimum value of the selected column.

Syntax:-

```sql
SELECT MIN(col_name) FROM table_name;
```

Ex:-

```sql
SELECT MIN(salary) FROM emp_info;
```

5. SUM() :- Returns total sum of selected numerical column.

Syntax:-

```sql
```

```sql
SELECT SUM(salary) FROM emp_info;
```

GROUP BY Clause :-

→ The 'group by' statement is used along with the select statement for organizing similar data into groups.
→ It is used to summarize data from the database.

Note:-
→ The 'select' statement is used with the 'groupby' clause in SQL queries.
→ WHERE clause is placed before GROUP BY clause.
→ ORDER BY clause is placed after GROUP BY clause.

Syntax:-

sql

```sql
SELECT col1, function(col2), ...
FROM table_name
WHERE condition
GROUP BY col1, col2
ORDER BY col_name (ASC | DESC);
```

Ex:-

sql

```sql
SELECT city, SUM(salary) FROM emp_info GROUP BY city;
```

HAVING Clause :-

→ HAVING clause places the condition in the groups defined by the GROUP BY clause.
→ 'WHERE' clause places conditions on selected columns, whereas the 'HAVING' clauses places conditions on groups created by 'GROUP BY' clause.

Syntax:-

sql

```sql
SELECT col1, col2, Agg_fun(col3), ...
FROM table_name
WHERE condition
GROUP BY col_name
HAVING condition;
```

Ex:-

```sql
SELECT SUM(salary), city FROM emp_info
GROUP BY city
HAVING SUM(salary) > 10000;
```

Sub-Queries :-

→ A subquery is a SELECT statement which is used in another SELECT statement.
→ Subqueries are useful when you need to select rows from a table with a condition that depends on the data of the table itself.
→ Also referred as nested select, sub select, inner select.
→ In general the subqueries executed first and its output is used in the main query or outer query.

Guidelines for using a Subquery :-

1. Enclose subqueries in ().
2. Place subqueries on the right side of the comparison operator.
3. Do not add 'ORDER BY' clause to a subquery.
4. Use single row operators with single row subqueries. `(<, >, <=, >=, <>)`
5. Use multiple-row operators with multiple row subqueries. `(IN, ANY, ALL)`
6. We can write a sub query within a subquery.

Types of Subqueries :-

1. Single row Subqueries :- → The subquery returns only one row, i.e it returns only one row from the Inner Select statement. → Uses single row comparison operators like `=, >, <, >=, <>`

2. Multiple row Subqueries :- → The subquery returns more than one row, i.e it returns more than one row from the Inner Select statement. → Uses multiple row comparison operators like `IN, ANY, ALL`

Single row Subqueries:- (example)

```sql
SELECT first_name, second_name FROM employees
WHERE office_code = (SELECT office_code FROM employees
                     WHERE first_name = 'George');
```

Multiple row Sub queries:-

```sql
SELECT f_Name, S_Name FROM employees WHERE office_code
IN (SELECT office_code FROM employees WHERE f_name IN ('TOM', 'Martin'));
```

Subqueries with ANY :- `>Any` → More than minimum. `<Any` → Less than maximum.

```sql
SELECT f_name FROM employees WHERE salary >
    Any (SELECT salary FROM employees WHERE dept_no=5);
```

Subqueries with ALL :- `>ALL` → More than Maximum. `<ALL` → Less than Minimum.

```sql
SELECT f_name FROM employees WHERE salary >
    ALL (SELECT salary FROM employees WHERE dept_no=5);
```

DML with Subqueries :-

1. Subqueries with INSERT:-

```sql
INSERT INTO emp_new SELECT * FROM emp_info WHERE ID IN (SELECT ID FROM emp_info);
```

2. Subqueries with UPDATE:-

```sql
UPDATE EMPLOYEE SET SALARY = salary * 0.25
WHERE AGE IN (SELECT AGE FROM customers WHERE AGE >= 29);
```

3. Subqueries with DELETE:-

```sql
DELETE FROM EMPLOYEE WHERE AGE IN (SELECT AGE FROM EMPLOYEE WHERE AGE >= 29);
```

JOINS :-

A SQL JOIN statement is used to combine data or rows from two or more tables based on a common field between them. Different types of joins are:

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN
5. CROSS JOIN
6. SELF JOIN

1. INNER JOIN :-

An INNER JOIN returns only the rows where there is a match in both tables.

Syntax:-

```sql
SELECT table1.col, table2.col2
FROM table1 INNER JOIN table2
ON table1.common_column = table2.common_column;
```

2. LEFT JOIN (LEFT OUTER JOIN) :-

It returns all rows from the left table, and the matching rows from the right table. If no match is found, NULL is returned for the right table's column.

Syntax:-

```sql
SELECT Table1.col1, Table2.col2
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

3. RIGHT JOIN (RIGHT OUTER JOIN) :-

It returns all rows from the right table, and the matching rows from the left table. If no match is found, NULL is returned for the left table's columns.

Syntax:-

```sql
SELECT table1.col1, table2.col2
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

4. FULL OUTER JOIN :-

It returns all rows when there is a match in either table. Rows with no match in one of the tables will have NULL in that table's columns.

Syntax:-

```sql
SELECT table1.col1, table2.col2
FROM table1
FULL OUTER JOIN table2
ON table1.common_column = table2.common_column;
```

5. CROSS JOIN :-

It returns the cartesian product of two tables, i.e every row from the first table is paired with every row from the second table.

Syntax:-

```sql
SELECT table1.col1, table2.col2
FROM table1
CROSS JOIN table2;
```

6. SELF JOIN :-

A SELF JOIN is a join in which a table is joined with itself.

Syntax:-

```sql
SELECT a.col1, b.col2
FROM table_name a, table_name b
WHERE condition;
```

Ex:-

```sql
SELECT el.Name AS Employee, e2.Name AS Manager
FROM Employees e1
LEFT JOIN Employees e2
ON e1.ManagerID = e2.EmpID;
```

VIEWS :-

→ Views in SQL are kind of virtual tables.
→ A view also has rows and columns as they are in a real table in the database.
→ We can create a view by selecting fields from one or more tables present in the database.
→ A view can either have all the rows of a table or specific rows based on certain condition.

Creating a View :-

```sql
CREATE VIEW view_name AS
SELECT col1, col2, ...
FROM table_name
WHERE condition;
```

Creating a view from single table:-

```sql
CREATE VIEW emp_view AS
SELECT name, manager_id FROM employees
WHERE emp_id <= 4;
```

Creating a view from multiple tables:-

```sql
```

```sql
CREATE VIEW employee_view AS
SELECT emp.manager_id, emp.name, emp_details.city, emp_details.age
FROM employee emp, emp_details
WHERE emp.name = emp_details.name;
```

Updating a view:-

```sql
UPDATE VIEW_Name SET name = 'xyz' WHERE manager_id = 4;
```

Deleting from a view:-

```sql
DELETE FROM view1 WHERE name = 'xyz';
```

Dropping a view:-

```sql
DROP VIEW view_name;
```

Foreign Key Referential Options :-

MySQL/PostgreSQL contains different referential options. These are as follows:-

1. CASCADE :- It is used when we delete or update any row from the parent table, the values of the matching rows in the child table will be deleted or updated automatically.

2. RESTRICT :- It is used when we delete or update any row from the parent table that has a matching row in the reference (child) table, PostgreSQL does not allow to delete or update rows in the parent table.

3. SET NULL :- If the referenced values in the parent table are deleted or modified all related values in the child table are set to null values.

4. NO ACTION :- The performed update or delete operation in the parent table will fail with an error.

Using Create table:-

```sql
```

```sql
CREATE TABLE child (
    id INT PRIMARY KEY,
    e_id INT,
    FOREIGN KEY (e_id) REFERENCES parent(id)
);
```

Using Alter table statement:-

```sql
ALTER TABLE contact ADD CONSTRAINT fk_person FOREIGN KEY (person_id)
REFERENCES person(ID) ON DELETE CASCADE ON UPDATE RESTRICT;
```

DROP foreign key:-

```sql
ALTER TABLE TABLE_NAME DROP CONSTRAINT fk_Name;
```

> **PostgreSQL Note:-** Use `DROP CONSTRAINT` instead of `DROP FOREIGN KEY`.

Scalar Functions :-

The scalar functions in SQL are used to return a single value from the given input value.

- `LOWER()` → Used to convert string column values to lowercase.
- `UPPER()` → To convert string column values to upper case.
- `SUBSTRING()` → Extracts substrings in SQL from column having string dtype.
- `ROUND()` → Rounds a numerical value to nearest integer.
- `NOW()` → To return current system date & time.
- `TO_CHAR()` → Used to format how a field must be displayed (PostgreSQL version of FORMAT()).

Comparison Functions :-

1. ISNULL(expr) :- (PostgreSQL uses `IS NULL` instead)

```sql
```

```sql
SELECT * FROM students WHERE age IS NULL;
```

2. GREATEST(val1, val2, ...) (or) LEAST(val1, val2, ...) :- Returns maximum value in case of greatest, minimum value in case of least function.

```sql
SELECT GREATEST(12, 1, 45);
SELECT LEAST(14, 1, 45);
```

3. COALESCE(expr1, expr2, expr3) :- (PostgreSQL's version of IF/IFNULL) Returns the first non-null value in the list.

```sql
SELECT COALESCE(NULL, NULL, 'Hello');  -- Returns 'Hello'
SELECT COALESCE(name, 'Unknown') FROM students;
```

4. NULLIF(expr1, expr2) :- Returns NULL if expr1 = expr2, otherwise returns expr1.

```sql
SELECT NULLIF(10, 10);  -- Returns NULL
SELECT NULLIF(10, 5);   -- Returns 10
```

Transactions (TCL) :-

A transaction is a unit of work. Either all steps succeed or none do (atomicity).

COMMIT :-

Saves all the changes made during the transaction permanently.

```sql
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

ROLLBACK :-

Undoes all changes made during the current transaction.

```sql
BEGIN;
DELETE FROM students WHERE id = 5;
ROLLBACK;  -- Student is NOT deleted
```

SAVEPOINT :-

Creates a point within a transaction to which you can later roll back.

```sql
BEGIN;
INSERT INTO students VALUES (1, 'Ravi', 20);
SAVEPOINT sp1;
INSERT INTO students VALUES (2, 'Priya', 22);
ROLLBACK TO sp1;  -- Priya's insert is undone
COMMIT;           -- Ravi's insert is saved
```

DCL — Data Control Language :-

GRANT :-

Used to give users access privileges to the database.

```sql
GRANT privilege_name ON object_name TO user_name;
```

Ex:-

```sql
GRANT SELECT ON students TO public;
GRANT ALL PRIVILEGES ON students TO john;
```

REVOKE :-

Used to remove previously granted access privileges.

```sql
REVOKE privilege_name ON object_name FROM user_name;
```

Ex:-

```sql
REVOKE SELECT ON students FROM john;
```

PostgreSQL Advanced Concepts

SEQUENCES :-

A sequence is a database object that generates unique integer values automatically. Used for auto-incrementing primary keys.

```sql
CREATE SEQUENCE seq_name
START WITH 1
INCREMENT BY 1
MINVALUE 1
MAXVALUE 1000
CYCLE;  -- restart from MINVALUE after MAXVALUE
```

Using a sequence:-

```sql
SELECT NEXTVAL('seq_name');   -- Get next value
SELECT CURRVAL('seq_name');   -- Get current value
SELECT SETVAL('seq_name', 5); -- Set to a specific value
```

Using SERIAL (shortcut for sequence + default):-

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50)
);
```

INDEXES :-

An index is a database object that speeds up data retrieval from a table. Think of it like a book index.

Creating an Index :-

```sql
CREATE INDEX index_name ON table_name (column_name);
```

Ex:-

```sql
CREATE INDEX idx_name ON students (name);
```

Types of Indexes in PostgreSQL :-

Type	Use Case
`B-TREE`	Default. For =, <, >, BETWEEN, LIKE 'abc%'
`HASH`	Only for = comparisons
`GIN`	For JSONB, Arrays, Full-text search
`GiST`	For geometric data, Full-text search
`BRIN`	For very large tables with sorted data

```sql
-- Multi-column index
CREATE INDEX idx_city_age ON students (city, age);

-- Unique index
CREATE UNIQUE INDEX idx_email ON users (email);

-- Partial index (only index some rows)
CREATE INDEX idx_active ON users (email) WHERE is_active = TRUE;
```

Dropping an Index :-

```sql
DROP INDEX index_name;
```

Checking Indexes :-

```sql
\di           -- List all indexes in psql
```

STORED PROCEDURES & FUNCTIONS :-

Creating a Function :-

```sql
CREATE OR REPLACE FUNCTION function_name(param1 datatype, param2 datatype)
RETURNS return_type AS $$
BEGIN
    -- function body
    RETURN value;
END;
$$ LANGUAGE plpgsql;
```

Ex:- Function to get full name:-

```sql
CREATE OR REPLACE FUNCTION get_full_name(first_name VARCHAR, last_name VARCHAR)
RETURNS VARCHAR AS $$
BEGIN
    RETURN first_name || ' ' || last_name;
END;
$$ LANGUAGE plpgsql;

-- Calling the function
SELECT get_full_name('John', 'Doe');
```

Ex:- Function with SELECT:-

```sql
CREATE OR REPLACE FUNCTION get_count()
RETURNS INT AS $$
DECLARE
    total INT;
BEGIN
    SELECT COUNT(*) INTO total FROM students;
    RETURN total;
END;
$$ LANGUAGE plpgsql;

SELECT get_count();
```

Dropping a Function :-

```sql
DROP FUNCTION function_name(param_types);
```

TRIGGERS :-

A trigger is a function that is automatically executed when a specific event (INSERT, UPDATE, DELETE) occurs on a table.

Step 1 — Create the trigger function :-

```sql
CREATE OR REPLACE FUNCTION trigger_function_name()
RETURNS TRIGGER AS $$
BEGIN
    -- NEW = new row, OLD = old row
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Step 2 — Create the trigger :-

```sql
```

```
CREATE TRIGGER trigger_name
BEFORE INSERT OR UPDATE ON table_name
FOR EACH ROW
EXECUTE FUNCTION trigger_function_name();
```

Ex:- Auto-update timestamp on update:-

```sql
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER set_timestamp
BEFORE UPDATE ON students
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();
```

Trigger Types :-

Timing	Event	Level
BEFORE	INSERT	FOR EACH ROW
AFTER	UPDATE	FOR EACH STATEMENT
INSTEAD OF	DELETE	—

Dropping a Trigger :-

```sql
DROP TRIGGER trigger_name ON table_name;
```

WINDOW FUNCTIONS :-

Window functions perform calculations across a set of rows related to the current row, without

collapsing rows (unlike GROUP BY).

Syntax:-

```sql
function_name() OVER (
    PARTITION BY column
    ORDER BY column
    ROWS/RANGE frame_spec
)
```

ROW_NUMBER() :-

Assigns a unique sequential number to each row.

```sql
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank
FROM emp_info;
```

RANK() :-

Assigns rank with gaps for ties.

```sql
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rank
FROM emp_info;
```

DENSE_RANK() :-

Assigns rank without gaps for ties.

```sql
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
FROM emp_info;
```

LAG() / LEAD() :-

Access values from previous or next rows.

```sql
SELECT name, salary,
       LAG(salary) OVER (ORDER BY id) AS prev_salary,
       LEAD(salary) OVER (ORDER BY id) AS next_salary
FROM emp_info;
```

SUM() / AVG() with PARTITION :-

```sql
SELECT name, city, salary,
       SUM(salary) OVER (PARTITION BY city) AS city_total
FROM emp_info;
```

NTILE(n) :-

Divides rows into n equal buckets.

```sql
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary) AS quartile
FROM emp_info;
```

CTEs — Common Table Expressions :-

A CTE is a temporary result set that can be referenced within a SELECT, INSERT, UPDATE, or DELETE statement.

Syntax:-

```sql
WITH cte_name AS (
    SELECT ...
)
SELECT * FROM cte_name;
```

Ex:-

```sql
```

```sql
WITH high_salary AS (
    SELECT name, salary FROM emp_info WHERE salary > 50000
)
SELECT * FROM high_salary ORDER BY salary DESC;
```

Recursive CTE :-

Used for hierarchical data (org charts, trees).

```sql
WITH RECURSIVE employee_hierarchy AS (
    -- Base case
    SELECT id, name, manager_id, 0 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    -- Recursive case
    SELECT e.id, e.name, e.manager_id, eh.level + 1
    FROM employees e
    JOIN employee_hierarchy eh ON e.manager_id = eh.id
)
SELECT * FROM employee_hierarchy;
```

JSON & JSONB in PostgreSQL :-

> PostgreSQL's superpower — native JSON support!

Storing JSON :-

```sql
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    details JSONB
);

INSERT INTO products (details)
VALUES ('{"name": "Laptop", "price": 80000, "specs": {"ram": "16GB"}}');
```

Querying JSON :-

```sql
```

```sql
-- Get value by key (returns JSON)
SELECT details -> 'name' FROM products;

-- Get value as text
SELECT details ->> 'name' FROM products;

-- Nested access
SELECT details -> 'specs' ->> 'ram' FROM products;

-- Filter by JSON field
SELECT * FROM products WHERE details ->> 'name' = 'Laptop';
```

JSON Operators :-

Operator	Description
`->`	Get JSON field (returns JSON)
`->>`	Get JSON field as text
`#>`	Get nested JSON by path
`#>>`	Get nested JSON as text
`@>`	Contains
`<@`	Contained by
`?`	Key exists?

sql

```sql
-- Does key 'price' exist?
SELECT * FROM products WHERE details ? 'price';

-- Contains a specific JSON
SELECT * FROM products WHERE details @> '{"name": "Laptop"}';
```

JSON Functions :-

sql

```sql
SELECT JSON_OBJECT_KEYS(details) FROM products;  -- All keys
SELECT JSONB_ARRAY_ELEMENTS(details->'tags') FROM products; -- Array elements
SELECT JSONB_BUILD_OBJECT('name', 'Ravi', 'age', 25); -- Build JSON
```

ARRAYS in PostgreSQL :-

> Another PostgreSQL superpower — columns can store arrays!

sql

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    scores INT[]
);

INSERT INTO students (name, scores) VALUES ('Ravi', '{90, 85, 92}');
INSERT INTO students (name, scores) VALUES ('Priya', ARRAY[88, 79, 95]);
```

Querying Arrays :-

sql

```sql
-- Get first element (arrays are 1-indexed!)
SELECT scores[1] FROM students;

-- Filter: array contains value
SELECT * FROM students WHERE 90 = ANY(scores);

-- All scores > 80
SELECT * FROM students WHERE scores > ALL(ARRAY[80]);

-- Unnest array into rows
SELECT name, UNNEST(scores) AS score FROM students;
```

Array Functions :-

sql

```sql
SELECT ARRAY_LENGTH(scores, 1) FROM students;   -- Length of 1st dimension
SELECT ARRAY_APPEND(scores, 100) FROM students; -- Add to end
SELECT ARRAY_REMOVE(scores, 85) FROM students;  -- Remove value
SELECT ARRAY_CAT(ARRAY[1,2], ARRAY[3,4]);       -- Concatenate arrays
```

FULL-TEXT SEARCH in PostgreSQL :-

Full-text search allows you to search through text documents intelligently.

sql

```sql
-- Convert text to tsvector (searchable form)
SELECT TO_TSVECTOR('The quick brown fox jumps over the lazy dog');

-- Convert query to tsquery
SELECT TO_TSQUERY('quick & fox');

-- Search: does document match query?
SELECT TO_TSVECTOR('The quick brown fox') @@ TO_TSQUERY('quick & fox');
```

Full-text search on a table :-

sql

```sql
CREATE TABLE articles (
    id SERIAL PRIMARY KEY,
    title TEXT,
    content TEXT,
    search_vector TSVECTOR
);

-- Index for fast search
CREATE INDEX idx_fts ON articles USING GIN(search_vector);

-- Search
SELECT title FROM articles
WHERE search_vector @@ TO_TSQUERY('postgresql & database');

-- Rank results
SELECT title,
       TS_RANK(search_vector, TO_TSQUERY('postgresql')) AS rank
FROM articles
WHERE search_vector @@ TO_TSQUERY('postgresql')
ORDER BY rank DESC;
```

TABLE INHERITANCE :-

> Unique to PostgreSQL — tables can inherit from other tables!

sql

```sql
CREATE TABLE person (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    age INT
);

CREATE TABLE student (
    grade VARCHAR(10)
) INHERITS (person);

CREATE TABLE employee (
    department VARCHAR(50),
    salary NUMERIC
) INHERITS (person);

INSERT INTO student (name, age, grade) VALUES ('Ravi', 20, 'A');
INSERT INTO employee (name, age, department, salary) VALUES ('Priya', 30, 'IT', 6000

-- Query parent to get all rows from all child tables
SELECT * FROM person;
```

SCHEMAS in PostgreSQL :-

A schema is a namespace inside a database. It organizes tables, views, functions etc.

sql

```sql
-- Create a schema
CREATE SCHEMA hr;

-- Create table in schema
CREATE TABLE hr.employees (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);

-- Query from schema
SELECT * FROM hr.employees;

-- Show current search path
SHOW search_path;

-- Set search path (so you don't need schema prefix)
SET search_path TO hr, public;
```

ROLES & USERS in PostgreSQL :-

In PostgreSQL, users and roles are the same thing. A role can have login privilege.

sql

```sql
-- Create a role (no login)
CREATE ROLE analyst;

-- Create a user (with login)
CREATE USER john WITH PASSWORD 'secret123';

-- Grant role to user
GRANT analyst TO john;

-- Grant privileges
GRANT SELECT, INSERT ON students TO john;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO john;

-- Revoke
REVOKE INSERT ON students FROM john;

-- Drop user
DROP USER john;

-- List roles
\du
```

PARTITIONING :-

Table partitioning splits a large table into smaller physical pieces. Useful for very large datasets.

Range Partitioning :-

sql

```sql
CREATE TABLE orders (
    id SERIAL,
    order_date DATE,
    amount NUMERIC
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2023 PARTITION OF orders
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE orders_2024 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

List Partitioning :-

```sql
CREATE TABLE customers (
    id SERIAL,
    city VARCHAR(50)
) PARTITION BY LIST (city);

CREATE TABLE customers_hyd PARTITION OF customers FOR VALUES IN ('Hyderabad');
CREATE TABLE customers_mum PARTITION OF customers FOR VALUES IN ('Mumbai');
```

COPY Command (Bulk Import/Export) :-

```sql
-- Export table to CSV
COPY students TO '/tmp/students.csv' DELIMITER ',' CSV HEADER;

-- Import from CSV
COPY students FROM '/tmp/students.csv' DELIMITER ',' CSV HEADER;

-- From psql client (no superuser needed)
\copy students TO 'students.csv' CSV HEADER;
\copy students FROM 'students.csv' CSV HEADER;
```

Performance Tips :-

1. Use EXPLAIN to understand query plans:-

```sql
EXPLAIN SELECT * FROM students WHERE age > 20;
EXPLAIN ANALYZE SELECT * FROM students WHERE age > 20;
```

2. Use indexes on columns used in WHERE, JOIN, ORDER BY.

3. Use VACUUM to reclaim storage:-

```sql
```

```
VACUUM students;
VACUUM ANALYZE students;
```

4. Use connection pooling (PgBouncer) for production apps.

5. Prefer JSONB over JSON (JSONB is indexed and faster to query).

6. Use CTEs for readability, but use subqueries or joins for performance in complex cases.

Quick Reference — PostgreSQL vs MySQL Syntax Differences

Concept	MySQL	PostgreSQL
Auto increment	`AUTO_INCREMENT`	`SERIAL` or `GENERATED ALWAYS AS IDENTITY`
String concat	`CONCAT()`	`||` operator or `CONCAT()`
Current date	`NOW()`	`NOW()` or `CURRENT_DATE`
Limit & offset	`LIMIT 5, 10`	`LIMIT 10 OFFSET 5`
Rename column	`CHANGE col newcol dtype`	`RENAME COLUMN old TO new`
Date format	`DATE_FORMAT()`	`TO_CHAR()`
If null	`IFNULL(a, b)`	`COALESCE(a, b)`
Drop foreign key	`DROP FOREIGN KEY`	`DROP CONSTRAINT`
Modify column type	`MODIFY col dtype`	`ALTER COLUMN col TYPE dtype`
Boolean	`TINYINT(1)`	`BOOLEAN`
Show tables	`SHOW TABLES`	`\dt`
Describe table	`DESCRIBE tablename`	`\d tablename`

End of PostgreSQL Complete Notes